Laboratório de Concorrência · Aula 10

Luiz Otavio Roder · RA 24159236-2 · Turma ESOFT-C
Gerado em 19/10/2026, 19:07:05 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na cooperação, uma tarefa depende de outra para continuar: elas precisam se coordenar. Por exemplo, em um restaurante, o garçom espera o cozinheiro terminar o prato para poder entregá-lo. Na competição, duas tarefas disputam o mesmo recurso, que só pode ser usado por uma de cada vez. Por exemplo, duas pessoas precisam usar o mesmo caixa eletrônico e uma espera a outra terminar.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

A soma esperada é 2.000.000 (um milhão por thread). Porém, como as duas modificam o mesmo contador sem proteção, prevejo que o valor obtido possa ser menor e variar entre execuções.

1ª execução (n = 1.000.000)

1800261

2ª execução (n = 1.000.000)

1968625

3ª execução (n = 1.000.000)

1877223

Saída do programa

```
Execuções locais do código Java fornecido (os números podem variar):

n = 1.000.000, 1ª execução:
esperado: 2000000
obtido:   1800261

n = 1.000.000, 2ª execução:
esperado: 2000000
obtido:   1968625

n = 1.000.000, 3ª execução:
esperado: 2000000
obtido:   1877223

n = 1.000, 1ª execução:
esperado: 2000
obtido:   2000

n = 1.000, 2ª execução:
esperado: 2000
obtido:   2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não. O esperado era 2.000.000, mas os valores obtidos foram 1.800.261, 1.968.625 e 1.877.223. Eles não foram iguais. Isso acontece porque as threads são escalonadas em momentos diferentes e as operações de leitura, soma e escrita do contador não são atômicas: uma atualização pode sobrescrever a outra.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Suponha contador = 5. Primeiro, a thread A lê 5. Antes de A escrever, a thread B também lê 5. A soma 1 no seu registrador e obtém 6; B faz o mesmo e também obtém 6. A grava 6 na memória e depois B grava 6 de novo. Embora duas somas tenham sido realizadas, o contador passou de 5 para apenas 6, e não para 7. Uma atualização foi perdida.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, as duas execuções locais deram 2000, o valor correto. Isso não significa que o código seja seguro: menos operações reduzem a chance de ocorrer uma intercalação problemática, mas não a eliminam. Em um teste leve o problema pode passar despercebido; em produção, com mais carga e concorrência, ele pode causar perdas de atualização e resultados inconsistentes.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

BBBAAA → 7

ABABAB → 6

AABBBA → 4

AABABB → 6

Saída do programa

```
Execução local do programa Python fornecido:
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis, porque são seis passos no total, com três da tarefa A e três da tarefa B, mantendo-se a ordem interna de cada tarefa (C(6,3) = 20). Aparecem quatro resultados diferentes: 4, 6, 7 e 8. Apenas uma ordem produz o resultado considerado correto pelo enunciado, 8: AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

A ordem AABBBA produz 4. A lê TOTAL = 3 e calcula 3 + 1 = 4, mas ainda não grava. B lê o valor antigo 3, calcula 3 × 2 = 6 e grava 6. Por último, A grava o 4 que havia calculado. Assim, a atualização de B é sobrescrita e o resultado final fica 4, mesmo B tendo executado sua multiplicação.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A versão sem sincronização provavelmente será a mais rápida, porém pode apresentar um resultado incorreto. Espero que synchronized e AtomicInteger cheguem aos 2.000.000. Minha hipótese é que AtomicInteger tenha menos custo que synchronized nesse teste.

Saída do programa

```
Execuções locais do código Java fornecido:

1ª execução:
esperado:          2000000
sem sincronizacao: 1542004 em 12 ms
synchronized:      2000000 em 69 ms
AtomicInteger:     2000000 em 32 ms

2ª execução:
esperado:          2000000
sem sincronizacao: 1288353 em 9 ms
synchronized:      2000000 em 76 ms
AtomicInteger:     2000000 em 25 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com synchronized e AtomicInteger deram exatamente 2.000.000 nas duas execuções. A versão sem sincronização perdeu incrementos (1.542.004 e 1.288.353). Na primeira medição, os tempos foram 12 ms sem proteção, 69 ms com synchronized e 32 ms com AtomicInteger; na segunda, 9 ms, 76 ms e 25 ms. As versões corretas têm custo adicional de coordenação entre threads e acesso atômico, enquanto a incorreta não espera nem protege a seção crítica. Esses tempos dependem da máquina e do escalonamento.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo torna cada alteração individual atômica, mas não garante que o débito e o crédito ocorram como uma única transação. Outra operação poderia enxergar um estado intermediário (dinheiro retirado de uma conta, mas ainda não depositado na outra). É necessário proteger a operação inteira, por exemplo com uma transação bancária no banco de dados e bloqueios/controle de concorrência, garantindo atomicidade, consistência e rollback em caso de falha.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não espero que apareçam "A terminou" e "B terminou" no código original. A pode segurar X enquanto espera Y, e B pode segurar Y enquanto espera X. Como nenhuma libera seu primeiro recurso, o programa entra em impasse

(deadlock).

Saídas (antes e depois da correção)

```
Execuções locais do código Java fornecido:

Antes da correção:
IMPASSE: 2 threads presas

Após mudar B para adquirir X antes de Y:
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A tinha adquirido o bloqueio de X e aguardava o bloqueio de Y. Ao mesmo tempo, a thread B tinha adquirido Y e aguardava X. Nenhuma podia continuar até receber o segundo recurso, mas nenhuma liberava o primeiro. Essa espera circular manteve as duas presas em deadlock.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se todas as threads pegarem X antes de Y, uma delas poderá ficar esperando X, mas não segurará Y enquanto espera X. Isso impede que se forme a espera circular necessária ao impasse. Em um sistema bancário, duas transferências simultâneas entre contas poderiam se bloquear se cada uma travasse primeiro uma conta diferente; adquirir sempre as travas na ordem crescente do identificador das contas evita esse cenário.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com três trabalhadores, imagino que as seis tarefas de 100 ms sejam feitas em duas rodadas, somando cerca de 200 ms. Com um trabalhador, devem levar aproximadamente 600 ms; com seis, aproximadamente 100 ms. Há pequenas diferenças por causa do escalonamento e de outros custos.

Tempo com 1 trabalhador

610 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
Execuções locais do programa Go fornecido:

1 trabalhador:
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
```

```
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 610ms

3 trabalhadores:
trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 5
trabalhador 2 fez a tarefa 4
trabalhador 3 fez a tarefa 6
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

6 trabalhadores:
trabalhador 5 fez a tarefa 6
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Um trabalhador atende as seis tarefas em sequência e leva aproximadamente 600 ms (medi 610 ms). Três trabalhadores executam até três tarefas de cada vez, em duas rodadas, e levam cerca de 200 ms. Seis trabalhadores podem executar todas simultaneamente e terminam perto de 100 ms. O programa não incrementa uma variável compartilhada entre os trabalhadores: as tarefas são distribuídas por um canal, que coordena os envios e os recebimentos. O WaitGroup apenas espera que terminem. Assim, não é necessária uma trava para proteger um contador compartilhado nesse exemplo.

Saídas (antes e depois da troca)

```
Execuções locais do programa Go fornecido:

Canal sem buffer:
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /tmp/laboratorio_concorrencia_runs/p5b_bloqueio.go:9 +0x36
exit status 2

Após alterar para make(chan int, 1):
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No canal sem buffer, o envio ch <- 1 só termina quando outra goroutine recebe a mensagem. Como a única goroutine está parada tentando enviar, ela nunca chega ao fmt.Println que faria o recebimento. O Go identifica o deadlock. Com buffer de capacidade 1, o envio armazena a mensagem e prossegue sem precisar esperar um receptor naquele instante. Depois o próprio programa recebe e imprime 1.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Espero que o compilador Rust rejeite o programa, pois a closure enviada para thread::spawn captura uma variável local mutável que pertence à função main e não garante uma posse válida pela duração da thread. A correção deve exigir mudar a forma de capturar ou compartilhar o dado.

Saídas (o erro e o resultado com move)

```
Resultado previsto pela análise do código Rust (não executado nesta máquina):

Sem move:
error[E0373]: closure may outlive the current function, but it borrows `contador`
sugestão do compilador: usar `move` para transferir os valores capturados para a closure.

Com thread::spawn(move || ...):
0

Observação: o número inteiro é Copy; a thread altera sua própria cópia, e a variável de main
continua com 0.
```

Saída do programa

```
Resultado esperado para o código Rust com Arc<Mutex<i32>> (não executado nesta máquina):
contador = 400000

São quatro threads com 100.000 incrementos cada; o Mutex protege cada incremento.
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O tipo inteiro implementa Copy. Por isso, quando usamos move, o valor inicial do contador (0) é copiado para a closure da thread. A thread aumenta sua cópia para 1, mas a variável contador que ficou na função main continua com 0. Após o join, o println! de main imprime 0, porque não houve atualização compartilhada.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o programa sem sincronização compila normalmente e a condição de corrida só aparece durante a execução, quando o resultado pode sair incorreto. Em Rust, o compilador já rejeita o compartilhamento inseguro do primeiro exemplo, antes de rodar. A vantagem é detectar muitos problemas de posse e concorrência cedo, reduzindo erros em produção; o custo é aprender e escrever estruturas mais explícitas, como move, Arc e Mutex, e lidar com as regras do compilador.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Prevejo A, D, C e B. A e D são impressas imediatamente; C é uma microtarefa de Promise, executada após o código síncrono; B é uma tarefa agendada por setTimeout, mesmo tendo atraso de 0 ms.

Saída do programa

```
Execução local do JavaScript em Node.js:
A
D
C
B
E: timer de 10 ms rodou depois de 304 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem observada foi A, D, C, B. A e D são executadas diretamente no código principal. Depois que a pilha de execução fica livre, a microtarefa da Promise imprime C. Só então o laço de eventos executa o callback de setTimeout, que imprime B. O atraso 0 significa que a tarefa pode ser agendada o quanto antes, não que deva interromper o código atual.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer E, programado para 10 ms, só executou depois de cerca de 304 ms, porque ocupado(300) manteve a thread principal ocupada em um laço síncrono. O temporizador não pode executar enquanto o event loop está bloqueado. Em um servidor Node.js, um cálculo pesado desse tipo pode atrasar respostas a vários usuários. Uma saída é dividir o trabalho, usar workers ou transferir o cálculo para outro serviço/processo.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida: duas requisições leem que ainda existe uma vaga antes de qualquer uma atualizar o estoque e ambas concluem a compra, causando venda duplicada (overselling). Para evitar isso, a checagem e a reserva do último lugar devem formar uma operação atômica no servidor/banco de dados, usando transação, bloqueio de linha ou atualização condicional do estoque. Se apenas uma compra puder obter a reserva, a outra recebe a informação de indisponibilidade.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi ver que um programa pode parecer correto em alguns testes, mas perder resultados em outras execuções só pela ordem em que as threads trabalham.